CS241: Software Design and Implementation
FSTE / SCIMS

Final Examination
Semester 2 2017

Mode *(Face-to-Face)*

Total Marks: 100

Duration of Exam: 3 hours + 10 minutes

Reading Time: 10 minutes

Writing Time: 3 hours

Instructions:

There are three sections in this paper, with a total of 10 questions

This exam is worth 50 % of coursework

The minimum pass mark for this exam is 40 out of 100 marks

The question booklet consists of 4 pages

This is a closed book Exam

Write the answers in a separate answer booklet that will be distributed during the exam

All the questions are compulsory. There are no choices.

Section A (Short Answers) 30 marks

1. **A**. What is a corrective maintenance? Give an example. **(3 marks)**

Corrective maintenance refers to the process of fixing defects, bugs, or issues found in the software after it has been deployed. It aims to restore the software to its expected functioning when issues arise. This maintenance typically involves modifying the software to correct errors or unintended behavior that affects its performance or user experience.

Example:
If a user reports that a software application crashes when submitting a form, corrective maintenance would involve identifying the root cause (e.g., an issue with form validation) and fixing it so that the application no longer crashes.

B. Explain how adaptive maintenance applies to a software developed for ITS team at USP?

Adaptive maintenance involves updating the software to accommodate changes in the operating environment, such as updates to the operating system, hardware, or other dependencies. For the ITS (Information Technology Services) team at USP (University of the South Pacific), adaptive maintenance might involve modifying software to work with new versions of operating systems or changes in network configurations.

For instance, if the ITS team at USP develops a student management system that needs to work with a new version of a university-wide software platform or a new version of a database system, adaptive maintenance would involve making the necessary changes to ensure the system continues to operate smoothly in the new environment

2. **A**. What is alpha and beta release in software testing? Given an example for each release.

- **Alpha Release**:
  The alpha release is an early version of the software that is typically released to internal testers or developers for initial feedback and testing. It may contain incomplete features and is often used to identify major bugs and flaws in the system.
  Example:
  An early version of a mobile app, such as a new social media application, might be released internally to the development team to test core functionality, such as user login, profile creation, and messaging, before releasing it to a larger audience.

- **Beta Release**:
  The beta release is the version of the software that is ready for testing by a larger group of external users or public testers. It usually comes after the alpha release and aims to gather feedback on the overall user experience, as well as to identify less critical bugs.
  Example:
  A new video game might have a beta release where a selected group of users can play the game before it's officially launched. This helps developers identify issues related to user experience, game mechanics, and performance that weren't captured during the alpha phase.

B. What is acceptance testing? **(2 marks)**

Acceptance testing is the process of verifying that a software system meets the specified requirements and is ready for deployment. It is usually conducted by the customer or end-users to ensure the software functions as intended in real-world scenarios.

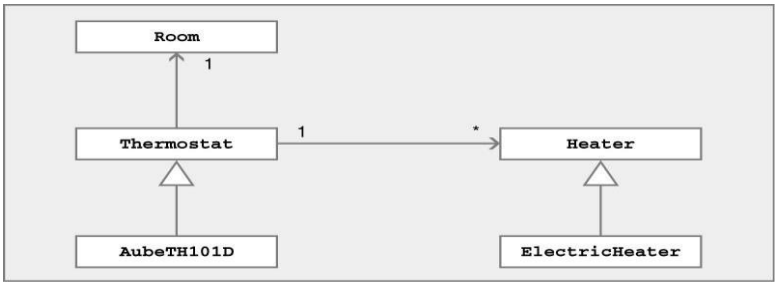
Example:
In a university management system, acceptance testing could involve the administrative staff testing the system to ensure that it correctly processes student enrollments, generates reports, and integrates with other university systems before it is approved for full use by the staff.

Section C (Case Study) 30 marks

3. **Software Design**

Observe the following UML diagrams and answer the questions with respect to each diagram in this section.

A. Given below is a simple class diagram of a Home Heating system. Explain the diagram with the relationships between each classes as illustrated in the diagram. **(5 marks)**



Classes and Relationships:

1. **Room:**

   Relationship: It has a one-to-one association with the **Thermostat** class, indicating each room is controlled by exactly one thermostat.

2. **Thermostat:**
   - **Relationships:**

     Room: One-to-one association with the Room class.

     Heater: One-to-many association with the Heater class, meaning a single thermostat can control multiple heaters.
   - **Inheritance:** It is a superclass for the **AubeTH101D** class.

3. **AubeTH101D:**
   - **Inheritance:** It inherits from the Thermostat class, indicating it shares common attributes and methods with Thermostat but also has additional specific features.

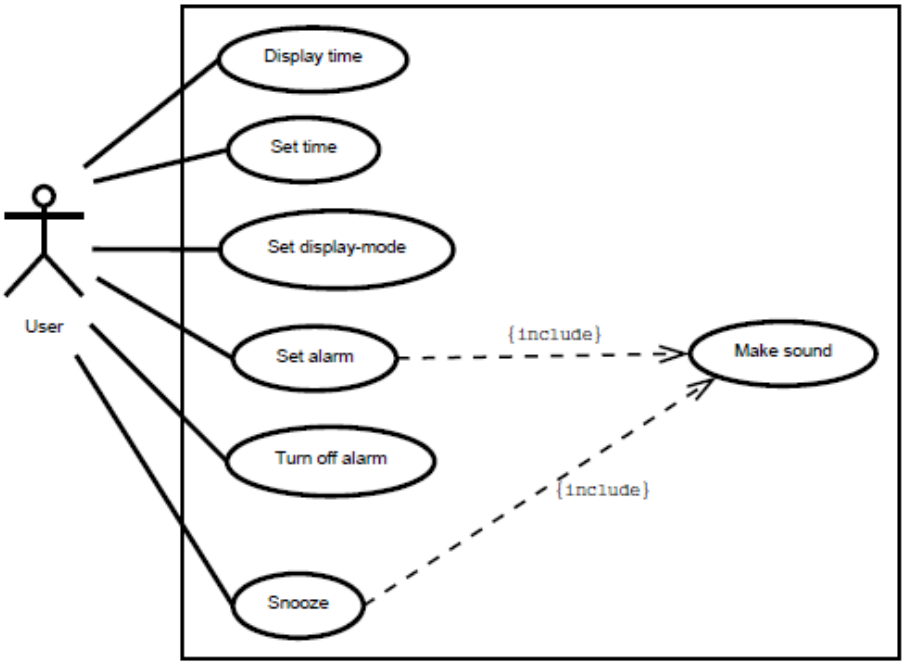
4. **Heater:**
   - **Relationship:** One-to-many association with the Thermostat class, indicating a single heater can be controlled by multiple thermostats.
   - **Inheritance:** It is a superclass for the **ElectricHeater** class.

5. **ElectricHeater:**
   - **Inheritance:** It inherits from the Heater class, suggesting it shares common attributes and methods with Heater but also has additional specific features.

B. For the following scenario draw a use case diagram. **(10 marks)**

Suppose we want to develop software for an alarm clock. The clock shows the time of day. Using buttons, the user can set the hours and minutes fields individually, and choose between 12 and 24-hour display. It is possible to set one or two alarms. When an alarm fires, it will sound some noise. The user can turn it off, or choose to 'snooze'. If the user does not respond at all, the alarm will turn off itself after 2 minutes. 'Snoozing' means to turn off the sound, but the alarm will fire again after some minutes of delay. This 'snoozing time' is pre-adjustable.

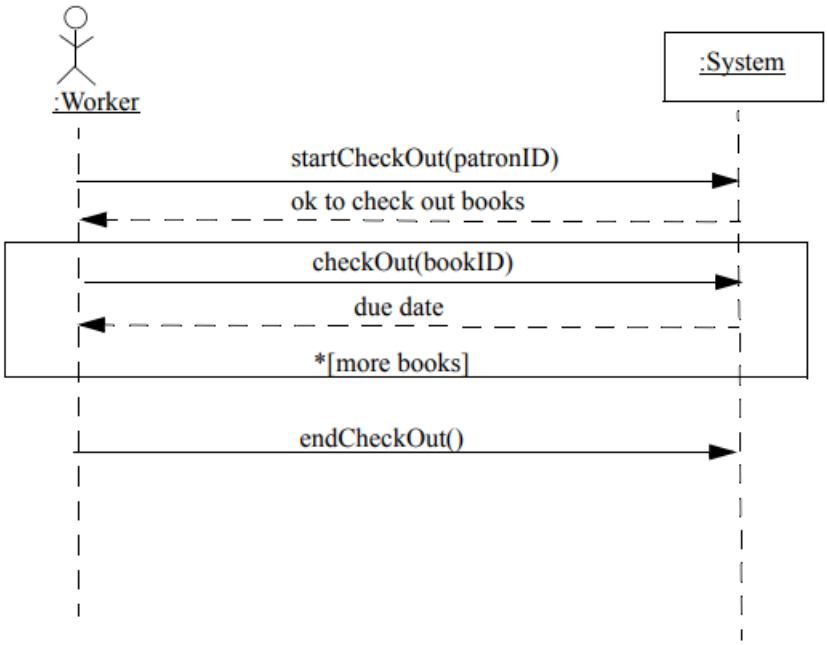
Answer:



C. Based on the use case diagram that you designed for **10 B**, provide necessary use case description for the use case: Snooze **(5 marks)**

D. Provided below is the basic flow obtained from a use case description. Draw a sequence diagram for the scenario. **(10 marks)**

- A Worker tells a System the identity of a patron who wishes to check out books.
- The System confirms that the patron is allowed to check out books, and remembers the patron's identity.
- The Worker tells the system the identity of a book this patron is checking out.
- The System confirms that the book can circulate, calculates the due date based on whether the patron is a faculty member or a student, and records that the patron has checked out this book, which is due on the calculated due date, and makes that information available from the library catalogue.
- The System tells the Worker the due date (which also confirms that the book has been checked out).
- The Worker repeats steps 3-5 until indicates done



THE END